



Computer Vision for Autonomous Vehicles

Correspondence and RANSAC

Instructor: Golnaz Habibi, PhD

University of Oklahoma

Fall 2024



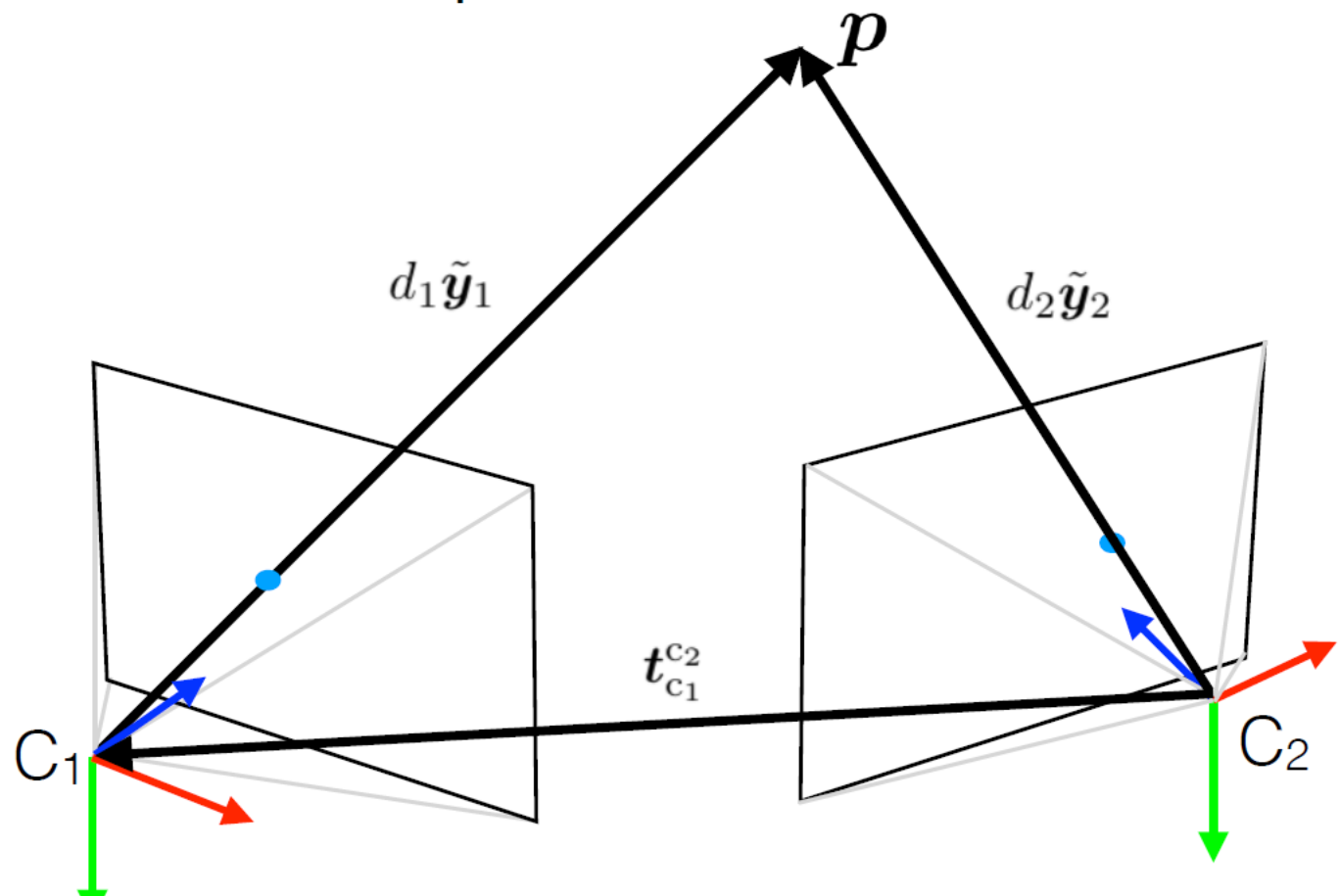
Announcement

- Lab 4 deadline is Oct 14, please start ASAP!

Re-cap: 2View Geometry

- Goal: Retrieve the camera positions using 2 views

Geometric Interpretation



$$d_1 \tilde{\mathbf{y}}_1 = \mathbf{p}^{C_1}$$

$$d_2 \tilde{\mathbf{y}}_2 = \mathbf{R}_{C_1}^{C_2} \mathbf{p}^{C_1} + \mathbf{t}_{C_1}^{C_2}$$

$$(\mathbf{t}_{C_1}^{C_2} \times \tilde{\mathbf{y}}_2) \perp (\mathbf{R}_{C_1}^{C_2} \tilde{\mathbf{y}}_1) \iff (\mathbf{t}_{C_1}^{C_2} \times \tilde{\mathbf{y}}_2)^\top \mathbf{R}_{C_1}^{C_2} \tilde{\mathbf{y}}_1 = 0$$

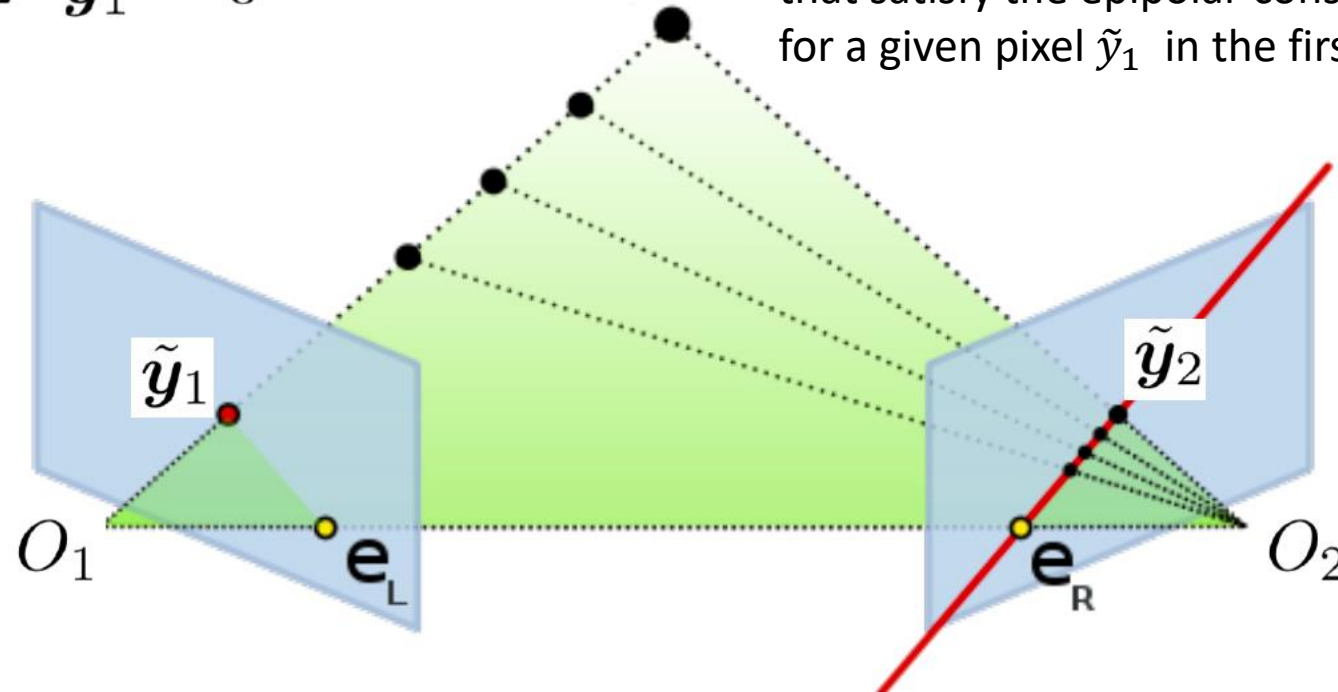
$$\iff ([\mathbf{t}_{C_1}^{C_2}]_\times \tilde{\mathbf{y}}_2)^\top \mathbf{R}_{C_1}^{C_2} \tilde{\mathbf{y}}_1 = 0 \iff \tilde{\mathbf{y}}_2^\top [\mathbf{t}_{C_1}^{C_2}]_\times \mathbf{R}_{C_1}^{C_2} \tilde{\mathbf{y}}_1 = 0$$

Epipolar Geometry

Epipolar Constraint:

$$\tilde{\mathbf{y}}_2^T \mathbf{E} \tilde{\mathbf{y}}_1 = 0$$

Epipolar line: Line corresponding to a set of pixels $\tilde{\mathbf{y}}_2$ in the second image that satisfy the epipolar constraints for a given pixel $\tilde{\mathbf{y}}_1$ in the first image



Plane passing through the optical centers and point p

 epipolar plane  epipolar line $\mathbf{e}_L, \mathbf{e}_R$: epipoles

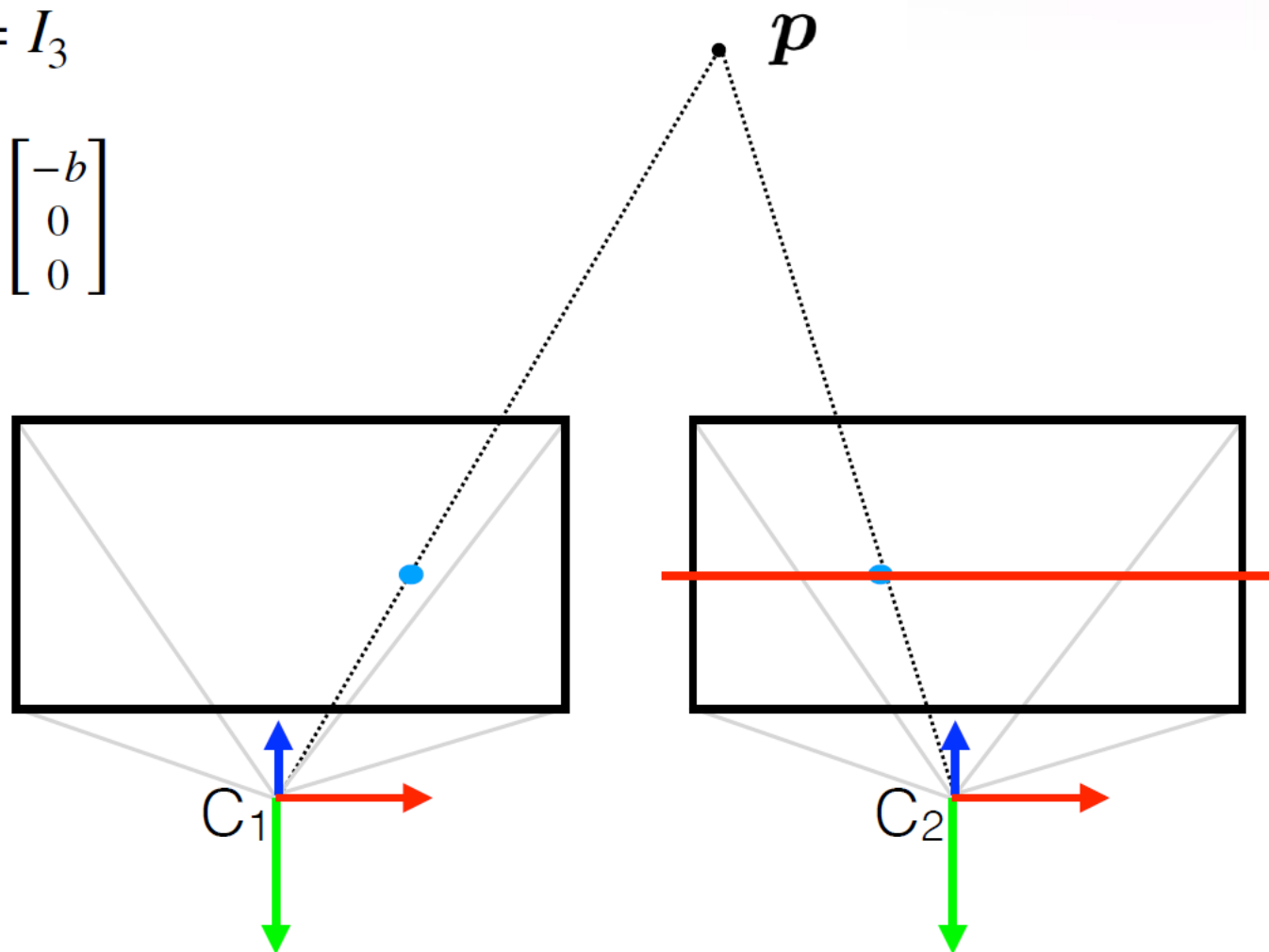
Plane passing through the optical centers and point p

Example: Stereo Camera



$$R_{C_1}^{C_2} = I_3$$

$$t_{C_1}^{C_2} = \begin{bmatrix} -b \\ 0 \\ 0 \end{bmatrix}$$



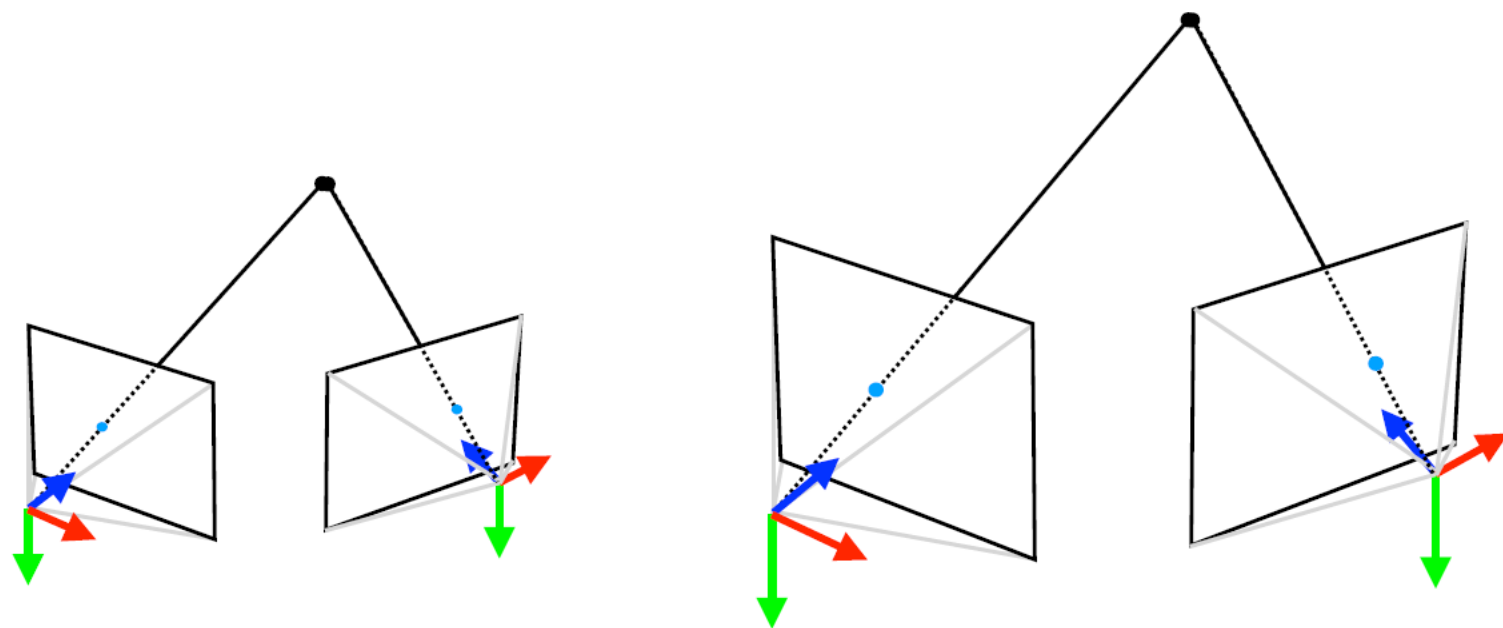
also: easy to triangulate points given geometry

Estimating Poses from Correspondences

Given N calibrated pixel correspondences:

$$(\tilde{\mathbf{y}}_{1,k}, \tilde{\mathbf{y}}_{2,k}) \text{ for } k = 1, \dots, N$$

compute the relative pose between the cameras



Can we estimate the scale of the translation (baseline)?

Estimating Poses from Correspondences

Given N calibrated pixel correspondences:

$$(\tilde{\mathbf{y}}_{1,k}, \tilde{\mathbf{y}}_{2,k}) \text{ for } k = 1, \dots, N$$

1. leverage the epipolar constraints to estimate the essential matrix $\mathbf{E}$

$$\tilde{\mathbf{y}}_{2,k}^T \mathbf{E} \tilde{\mathbf{y}}_{1,k} = 0$$

2. Retrieve the rotation and translation (up to scale) from the $\mathbf{E}$

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$$

Estimating Poses from Correspondences

Given N calibrated pixel correspondences:

$$(\tilde{\mathbf{y}}_{1,k}, \tilde{\mathbf{y}}_{2,k}) \text{ for } k = 1, \dots, N$$

1. leverage the epipolar constraints to estimate the essential matrix $\mathbf{E}$

$$\tilde{\mathbf{y}}_{2,k}^T \mathbf{E} \tilde{\mathbf{y}}_{1,k} = 0$$

2. Retrieve the rotation and translation (up to scale) from the $\mathbf{E}$

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$$

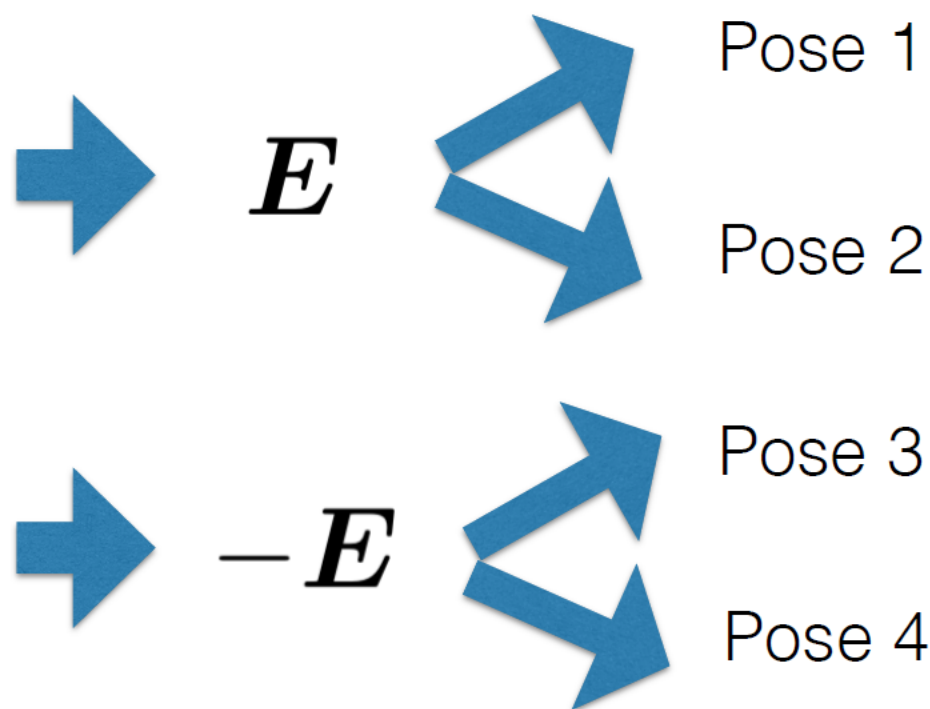
Retrieving Pose from Essential Matrix

Theorem 1 (Pose recovery from essential matrix, Thm 5.7 in [1]). *There exist exactly two relative poses $(\mathbf{R}, \mathbf{t})$ with $\mathbf{R} \in \text{SO}(3)$ and $\mathbf{t} \in \mathbb{R}^3$ corresponding to a nonzero essential matrix $\mathbf{E}$ (i.e., such that $\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$):*

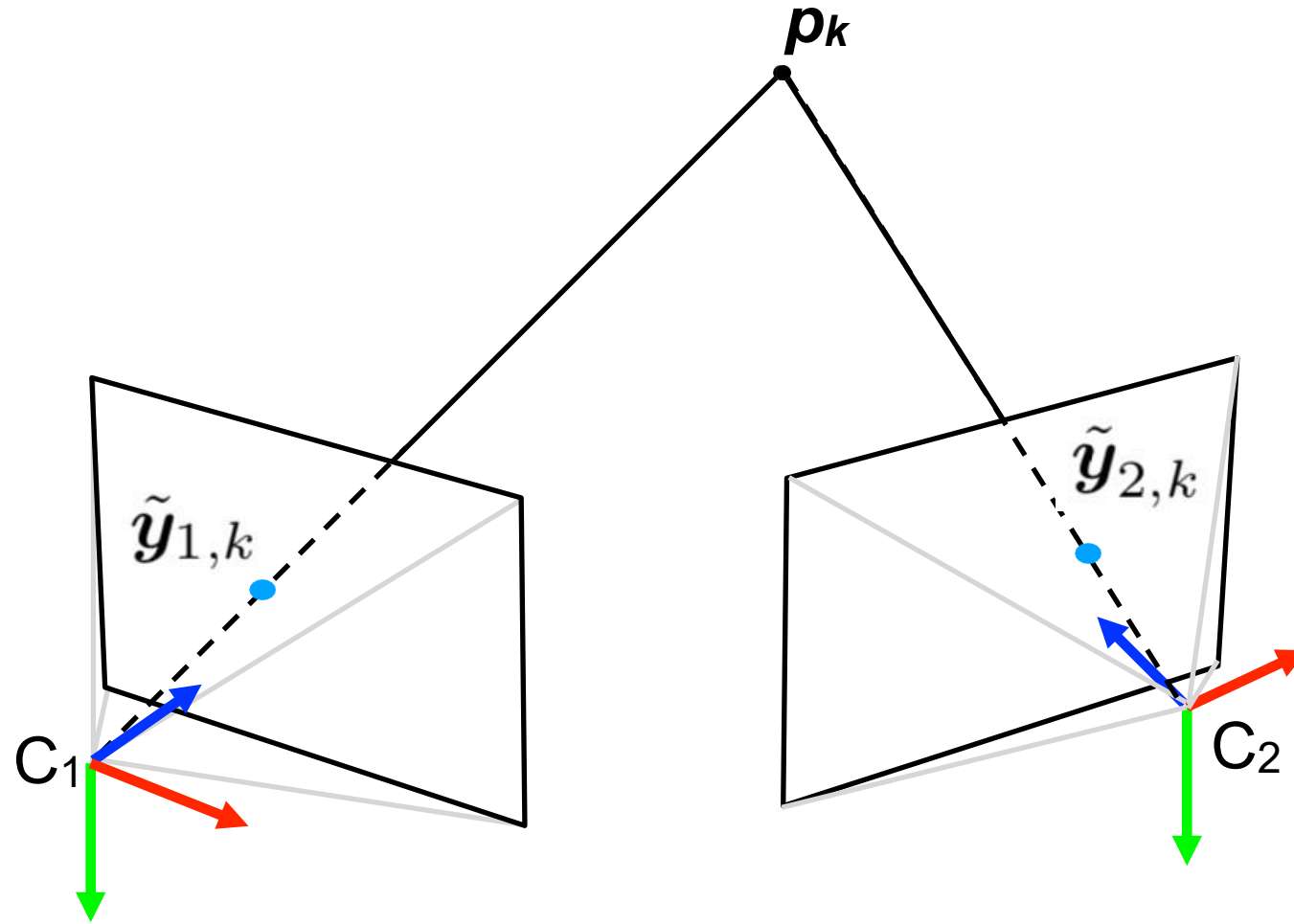
$$\mathbf{t}_1 = \mathbf{U} \mathbf{R}_z(+\pi/2) \mathbf{\Sigma} \mathbf{U}^{\top} \quad \mathbf{R}_1 = \mathbf{U} \mathbf{R}_z(+\pi/2) \mathbf{V}^{\top} \quad (13.19)$$

$$\mathbf{t}_2 = \mathbf{U} \mathbf{R}_z(-\pi/2) \mathbf{\Sigma} \mathbf{U}^{\top} \quad \mathbf{R}_2 = \mathbf{U} \mathbf{R}_z(-\pi/2) \mathbf{V}^{\top} \quad (13.20)$$

where $\mathbf{E} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^{\top}$ is the singular value decomposition of the matrix $\mathbf{E}$, and $\mathbf{R}_z(+\pi/2)$ is an elementary rotation around the z -axis of an angle $\pi/2$.



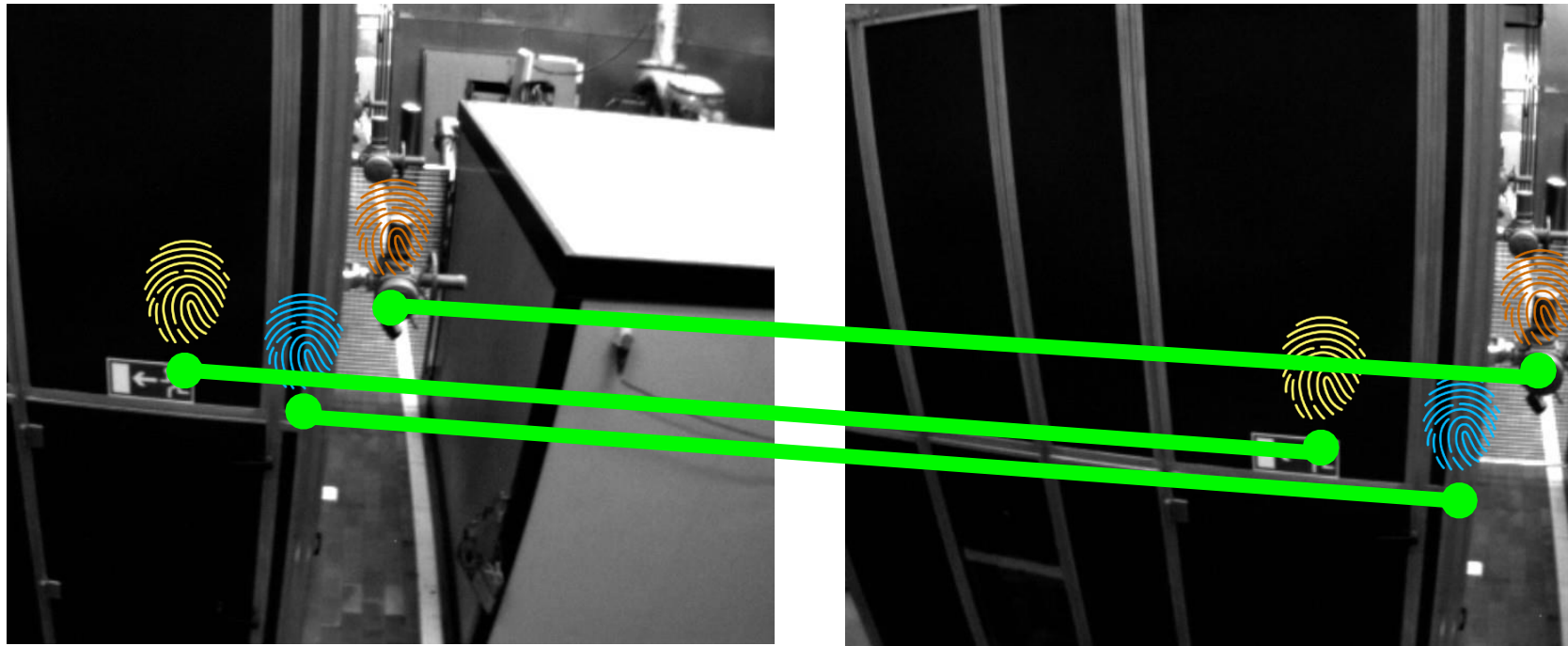
2-view Geometry



$$\tilde{y}_{2,k}^T E \tilde{y}_{1,k} = 0$$

Essential matrix encodes relative pose
(up to scale) between C_1 and C_2

2-view Geometry



**Last week's
assumptions:**

- no wrong correspondences (outliers)
- 3D point is not moving
- camera calibration is known

Estimating Poses from Correspondences

Given N calibrated pixel correspondences:

$$(\tilde{\mathbf{y}}_{1,k}, \tilde{\mathbf{y}}_{2,k}) \text{ for } k = 1, \dots, N$$

1. leverage the epipolar constraints to estimate the essential matrix $\mathbf{E}$

$$\tilde{\mathbf{y}}_{2,k}^T \mathbf{E} \tilde{\mathbf{y}}_{1,k} = 0$$

For 8 points: $\mathbf{A}\mathbf{e} = 0$ $N > 8$ points: $\arg \min_{\|\mathbf{e}\|=1} \|\mathbf{A}\mathbf{e}\|^2$

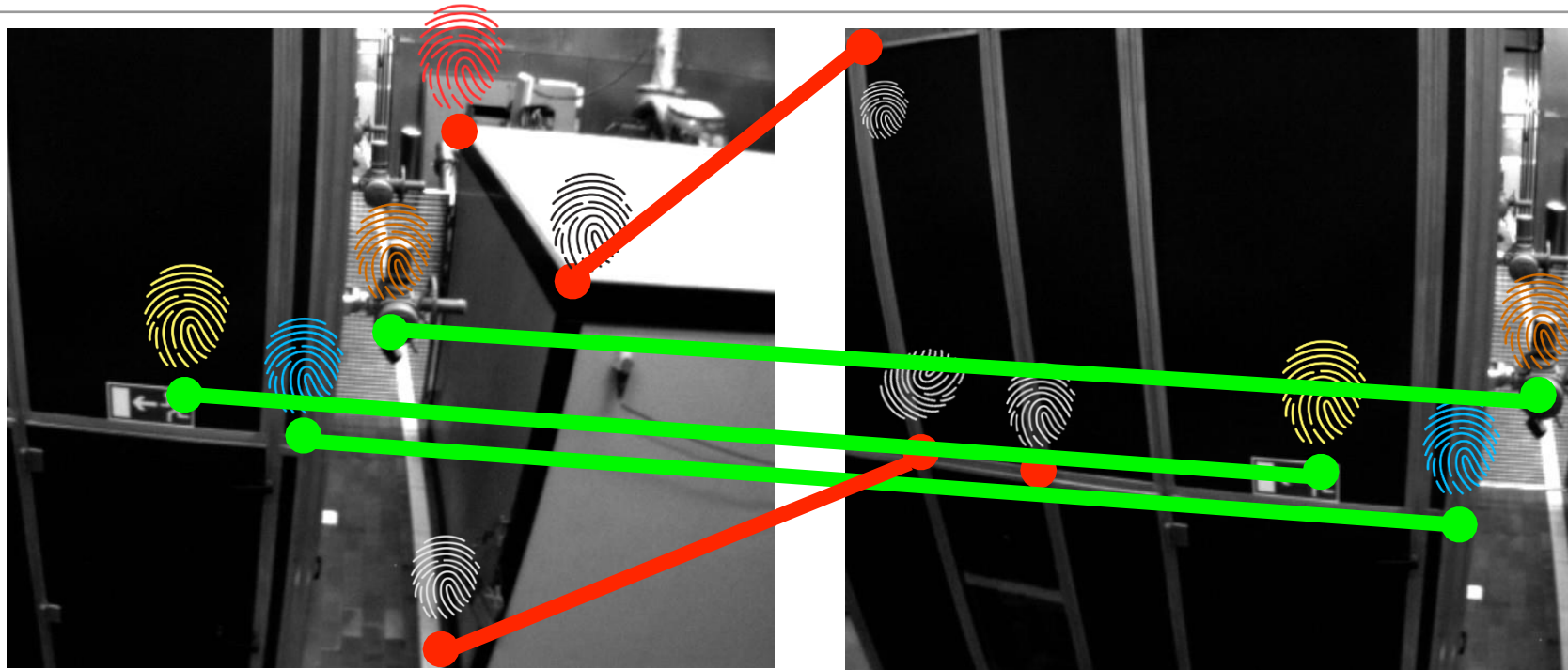
2. Retrieve the rotation and translation (up to scale) from the $\mathbf{E}$

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$$

How many correspondence do we need?

- Based on essential matrix (E) structure
 - Assume constraint on the translation vector $\|t\|^2 = 1$
 - t has two degree of freedom
 - R has three degree of freedom
 - We need at least 5 points
-
- Based on A in $Ae=0$ equation (i.e., Epipolar equation/constraints):
 - A is not full rank (it has one zero eigen value) $\rightarrow$ rank A is at most 8, we need 8 points.

2-view Geometry



- In practice:**
- Many wrong correspondences (outliers)
 - Some 3D points might be moving

Very important

- The camera estimation is very sensitive to the correspondence and we assume these correspondences are correct
- Issue: it is not in reality
- Solution: use algorithm to minimize the outlier such as RANSAC

RANSAC

RANdom **SA**mples **C**onsensus

Problem: estimate model P from N data points, possibly corrupted with outliers.

Assume: we have an algorithm to estimate P from n data points ($n \ll N$)

Basic idea:

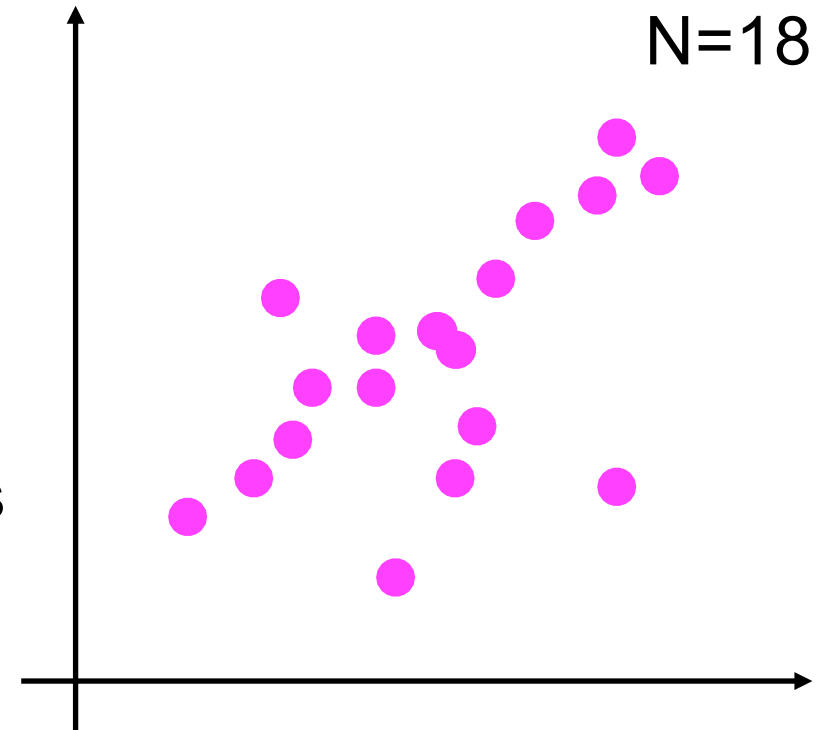
1. sample n points
2. compute an estimate P' of P
3. count how many other points agree with P'
4. repeat until you get a P' that agrees with many points



Example: Linear Regression

Fit a line through
N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points



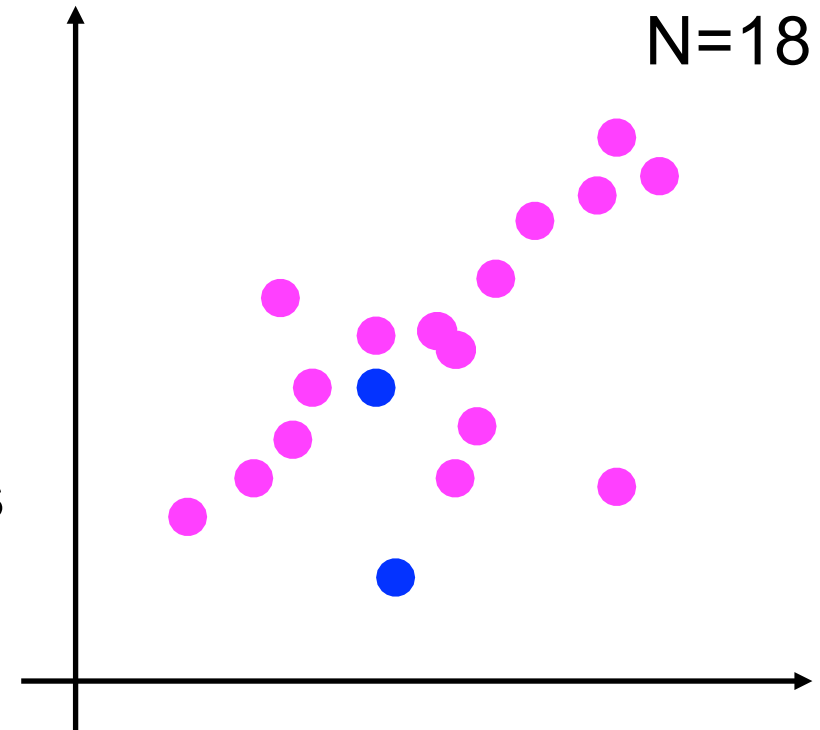
RANSAC:

1. sample 2 points
2. compute a line estimate P' of P
3. count how many points are within a **tolerance** from P'
4. repeat until you get a P' that agrees with many points

Example: Linear Regression

Fit a line through
N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points



RANSAC:

1.sample 2 points

2.compute a line estimate P' of P

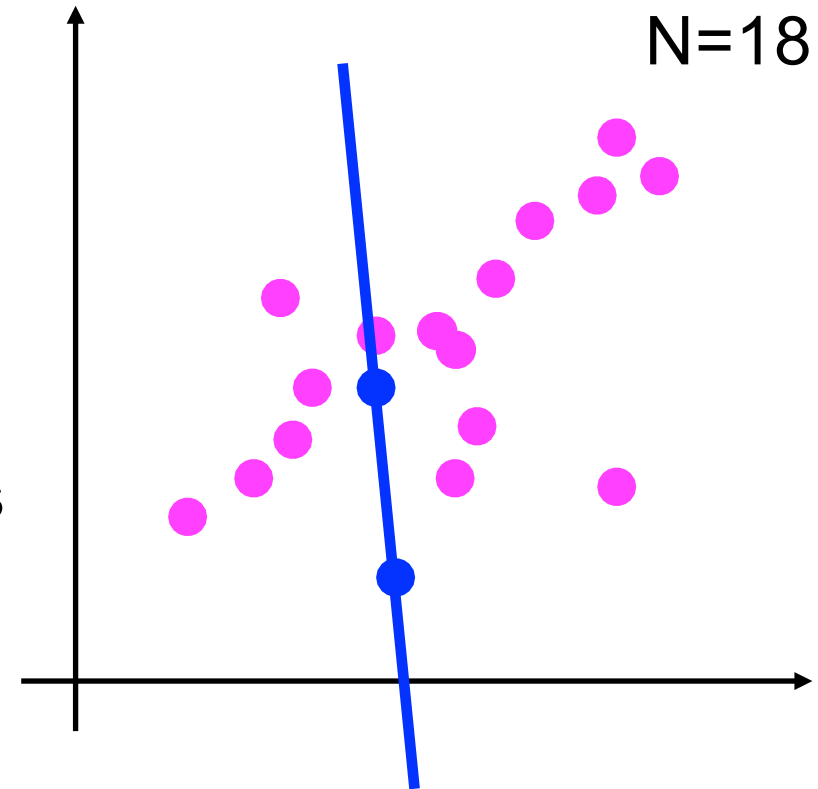
3.count how many points are within a **tolerance** from P'

4.repeat until you get a P' that agrees with many points

Example: Linear Regression

Fit a line through
N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points



RANSAC:

1. sample 2 points

2. compute a line estimate P' of P

3. count how many points are within a **tolerance** from P'

4. repeat until you get a P' that agrees with many points

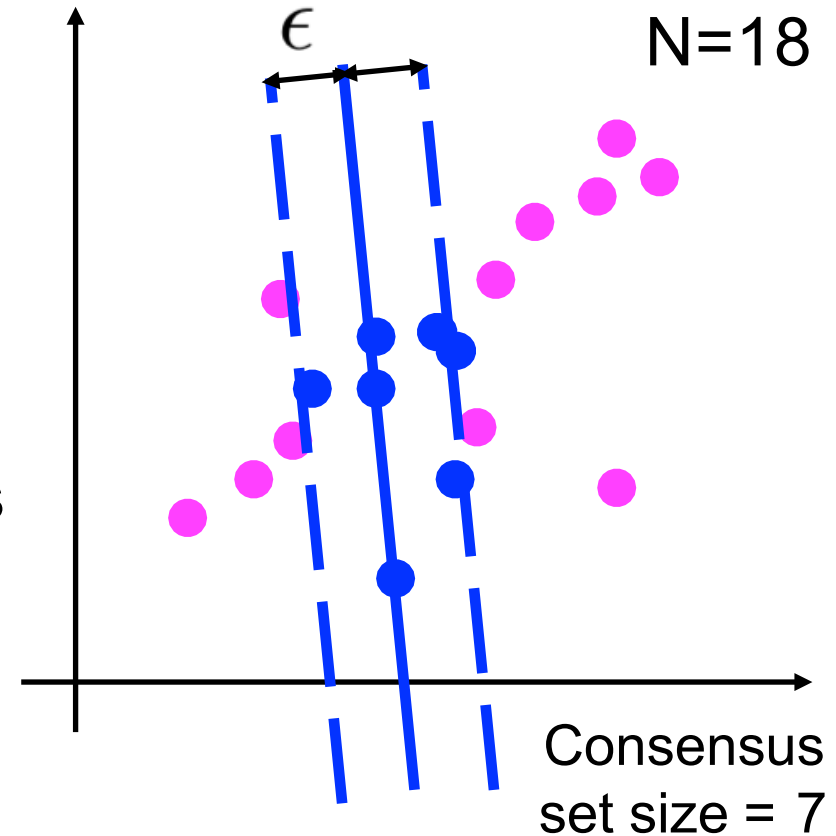
Example: Linear Regression

Fit a line through
 N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points

RANSAC:

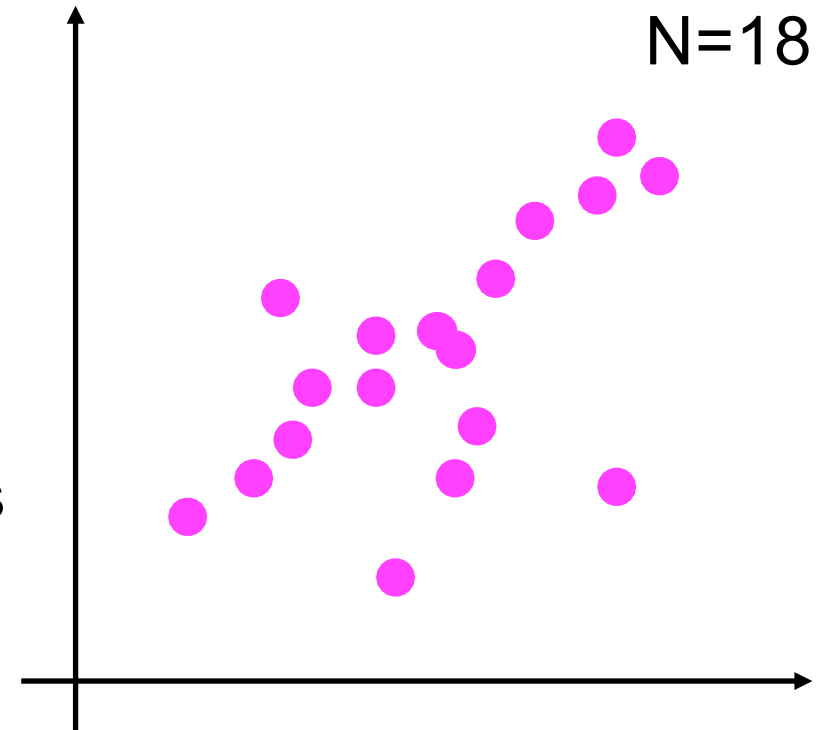
1. sample 2 points
2. compute a line estimate P' of P
3. count how many points are within a **tolerance** from P'
4. repeat until you get a P' that agrees with many points



Example: Linear Regression

Fit a line through
N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points



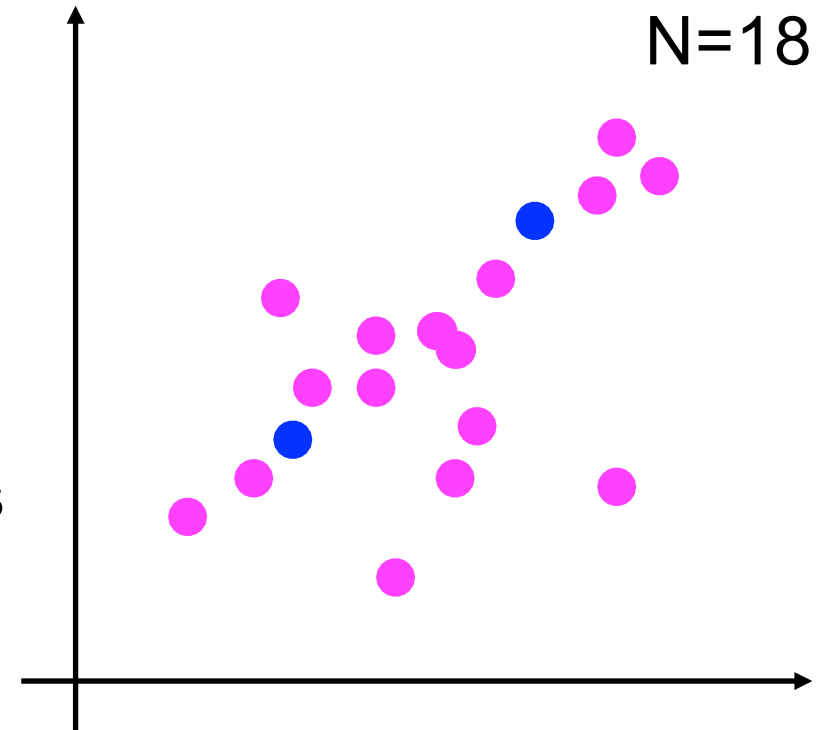
RANSAC:

1. sample 2 points
2. compute a line estimate P' of P
3. count how many points are within a **tolerance** from P'
4. repeat until you get a P' that agrees with many points

Example: Linear Regression

Fit a line through
N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points



RANSAC:

1.sample 2 points

2.compute a line estimate P' of P

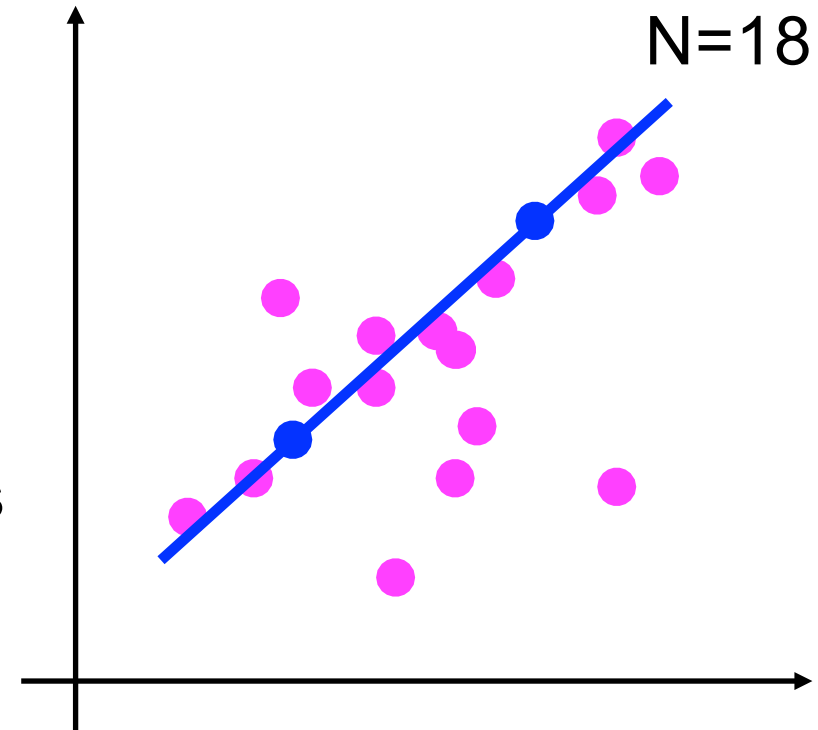
3.count how many points are within a **tolerance** from P'

4.repeat until you get a P' that agrees with many points

Example: Linear Regression

Fit a line through
N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points



RANSAC:

1. sample 2 points

2. compute a line estimate P' of P

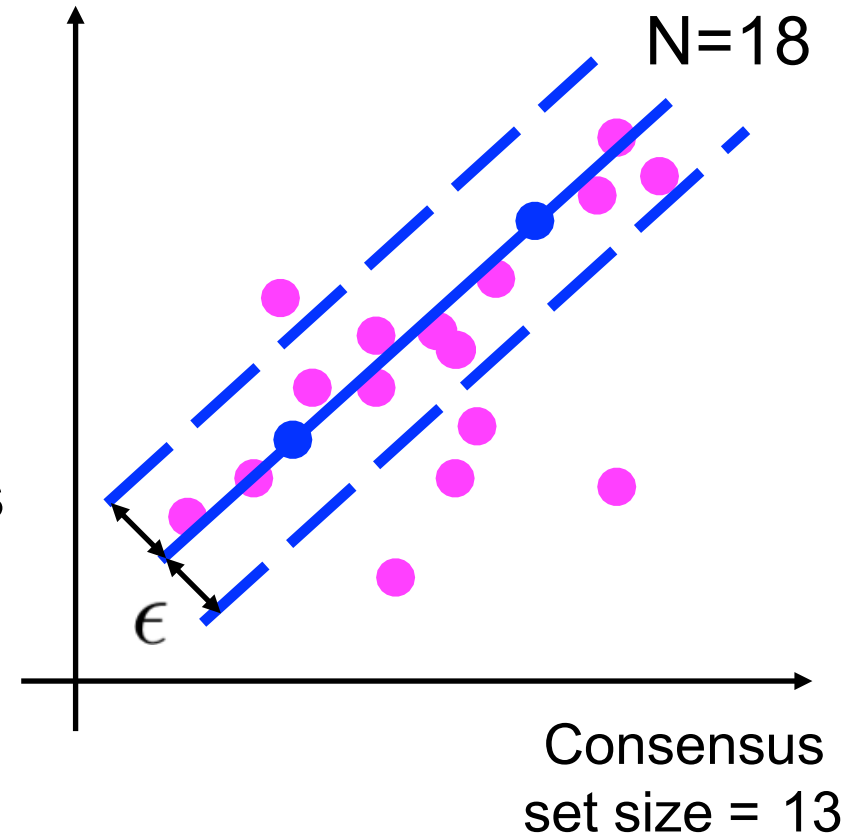
3. count how many points are within a **tolerance** from P'

4. repeat until you get a P' that agrees with many points

Example: Linear Regression

Fit a line through
 N 2D points, possibly
corrupted with outliers.

Note: we have an algorithm
to estimate a line from $n=2$ points



RANSAC:

1. sample 2 points

2. compute a line estimate P' of P

3. count how many points are within a **tolerance** from P'

4. repeat until you get a P' that agrees with many points

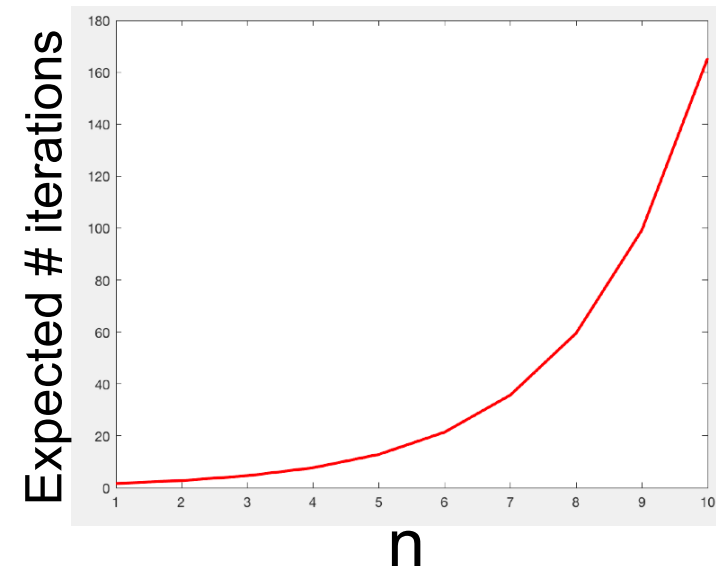
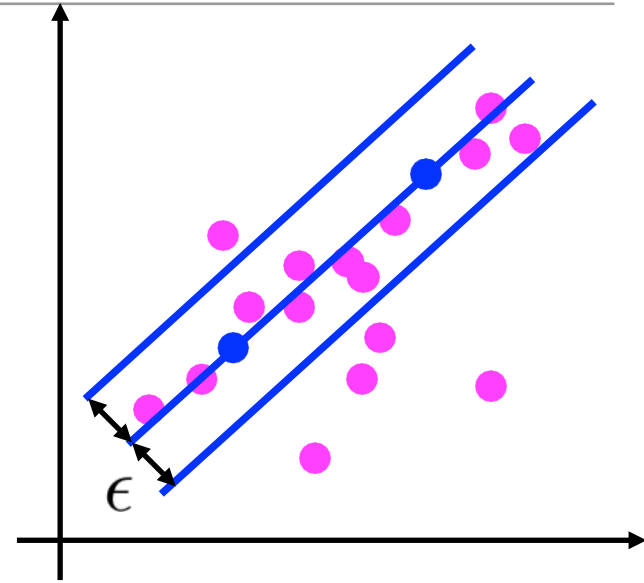
RANSAC: Parameter Tuning

1. Error Tolerance :
depends on the noise

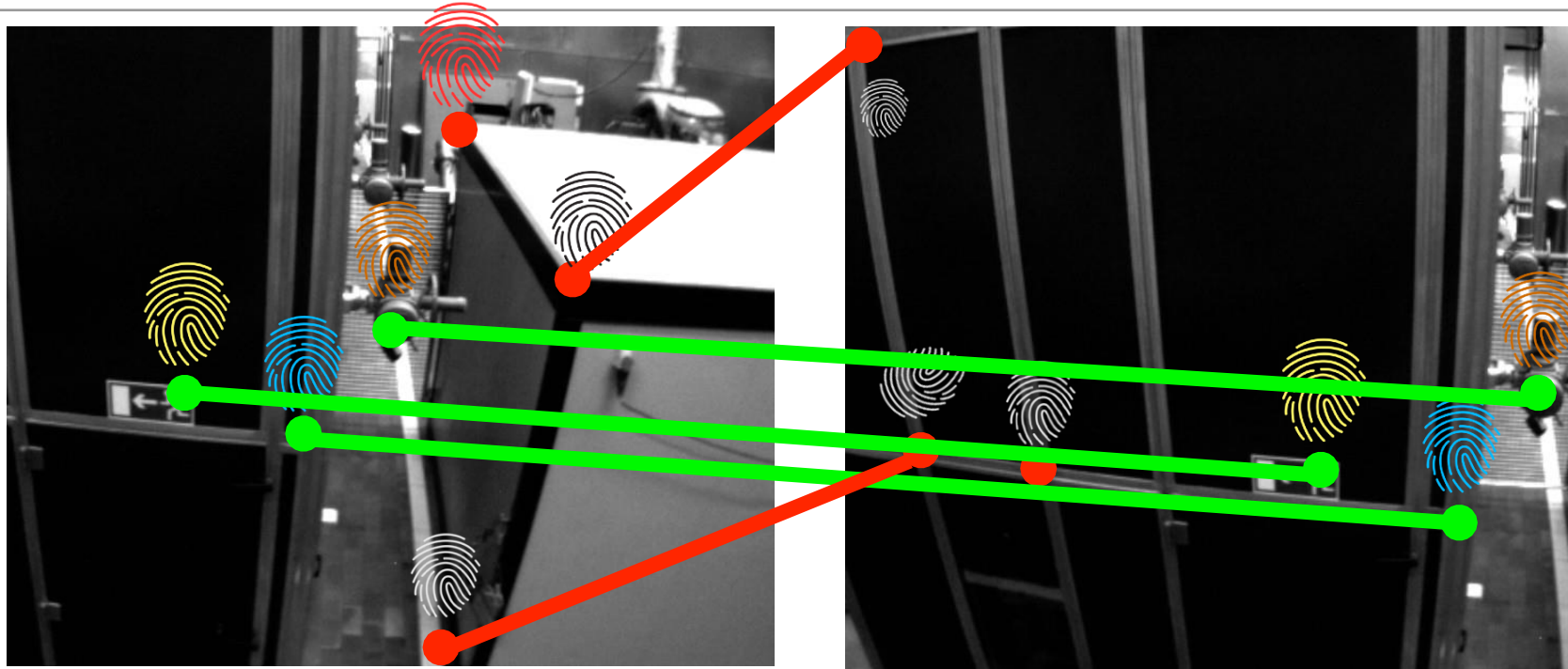
2. Acceptable consensus set:

- from the paper: $n+5$
- rule of thumb: $>50\%$ of points

3. Maximum number of iterations



Example: RANSAC for Essential Matrix estimation



RANSAC: $i=1,2 \quad \mathcal{F}_{i,k} \Rightarrow n \text{ sample} \rightarrow E$

1. sample n point correspondences $n \in S$

2. compute an estimate E' of the essential matrix E

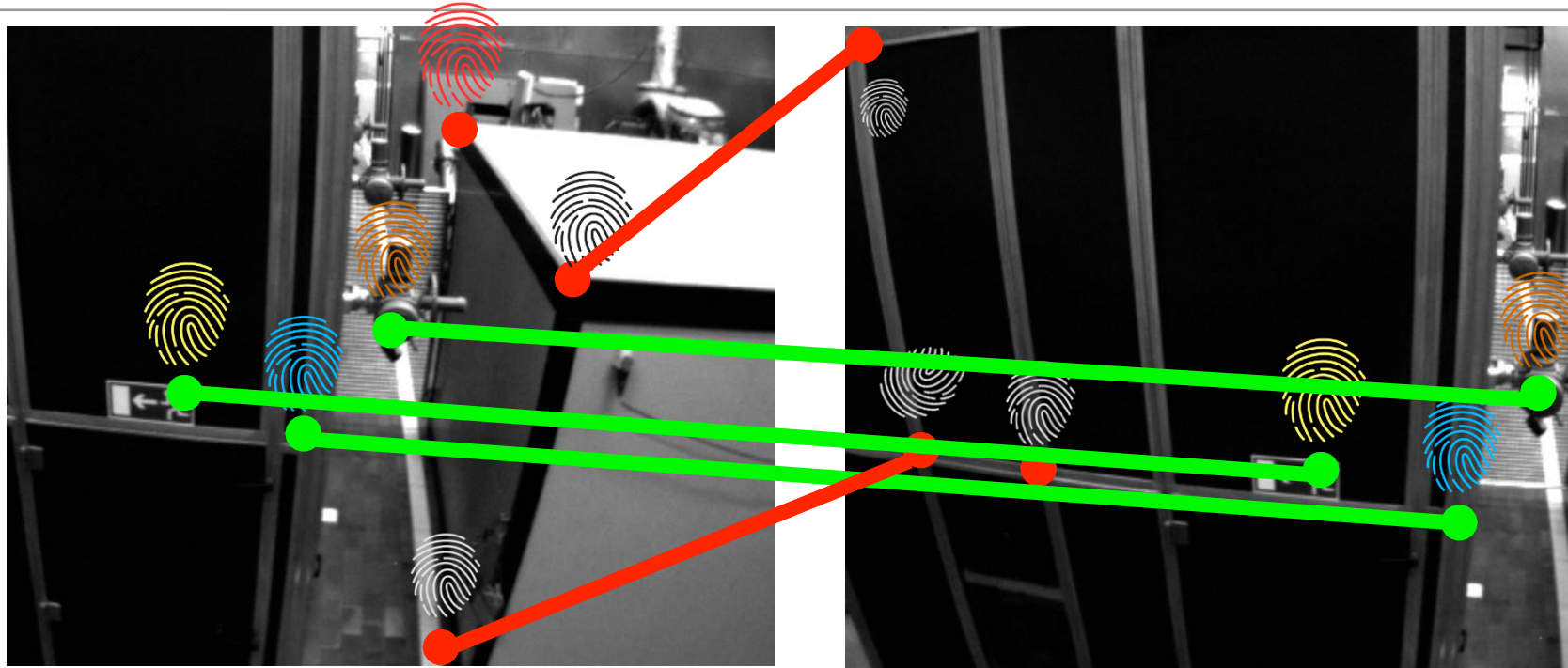
3. count how many points are within a **tolerance** from E'

4. repeat until you get a E' that agrees with many points

$$y_2^T E y_1 \approx \rho$$

$q \notin S$

Example: RANSAC for Essential Matrix estimation

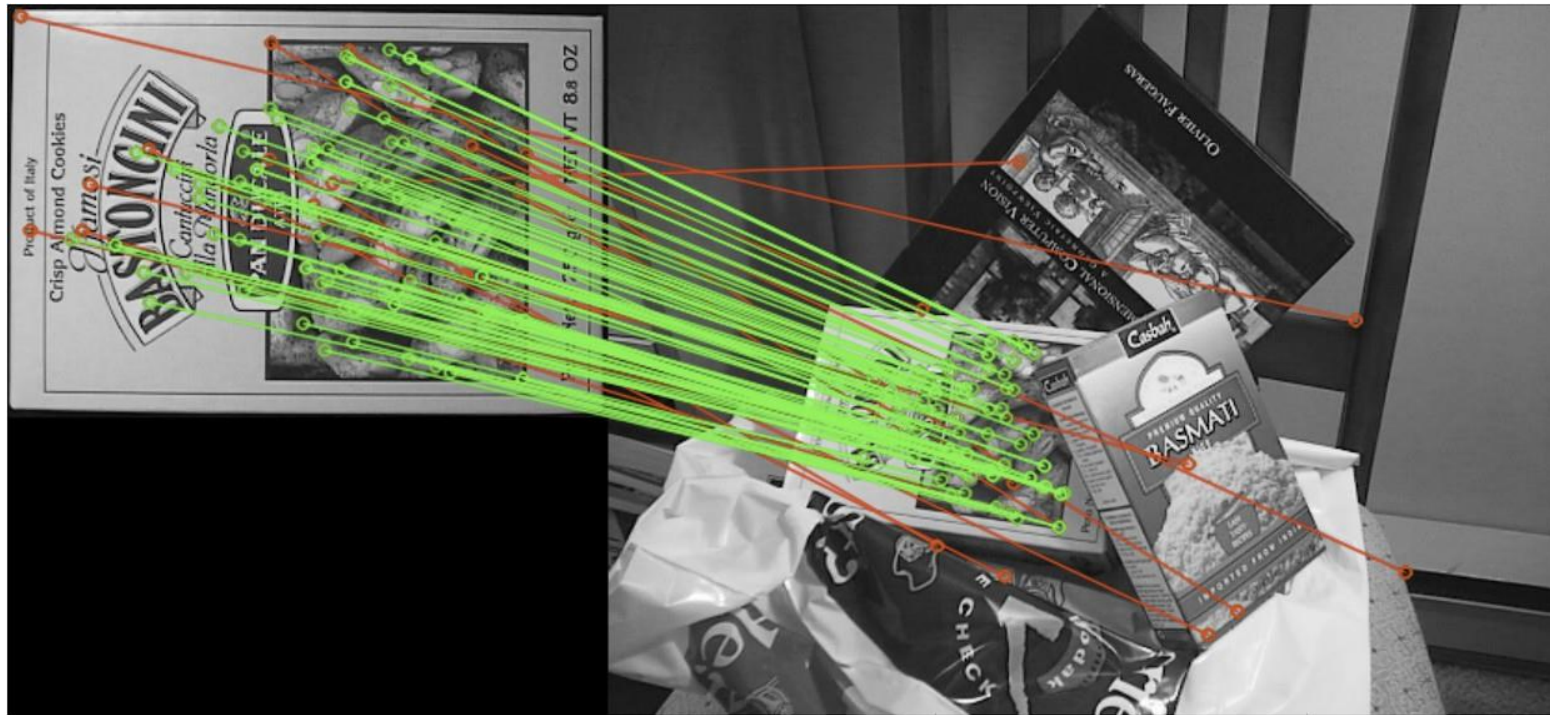


RANSAC

- essentially selects the set of inliers
- provides **geometric verification** for the correspondences

Beyond Motion Estimation

The tools we discussed (feature matching, essential matrix estimation, RANSAC) can be used also for **object detection and localization**



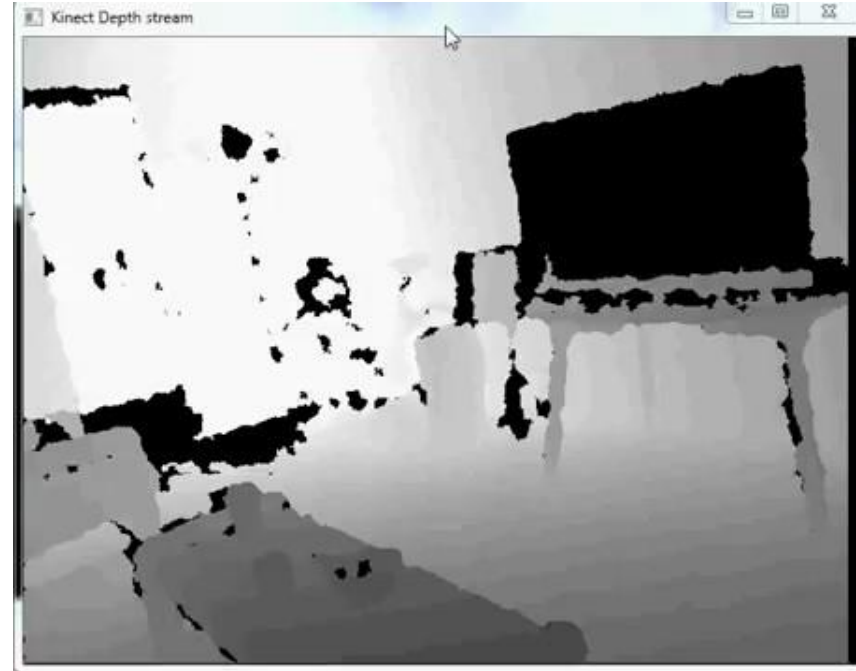
So far: pixel correspondences,
a.k.a., **2D-2D correspondences**

3D-3D Point Correspondences

Structured Light Cameras



© source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



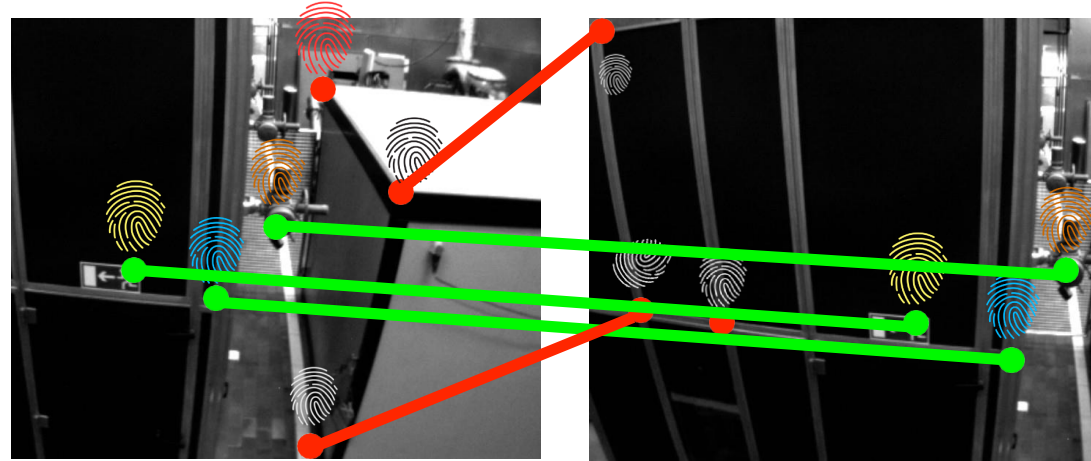
RGB-D cameras can measure depth (D) and image (RGB)

How can we use the depth information to estimate the relative pose between two RGB-D cameras observing the same scene?

3D-3D Point Correspondences

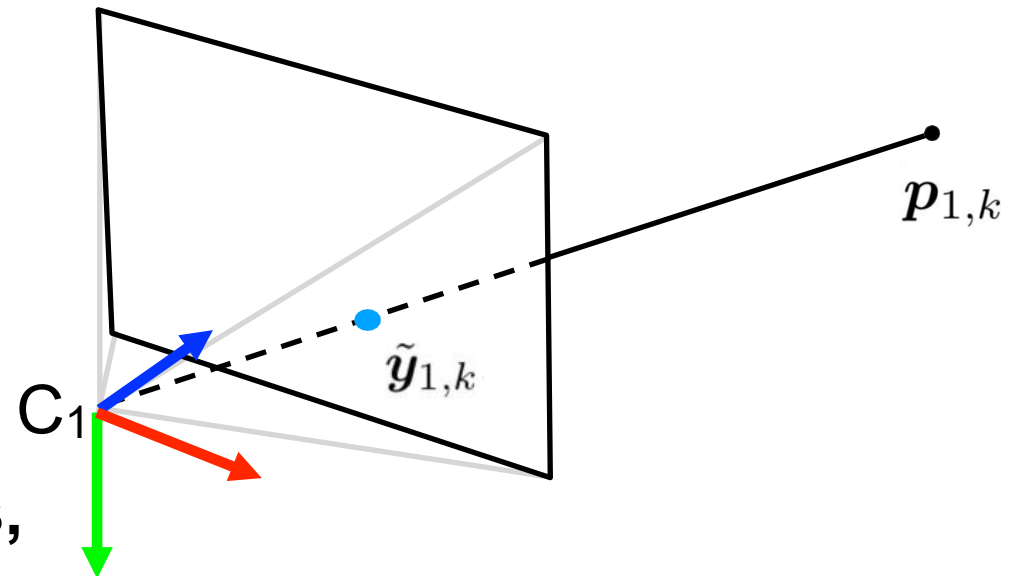
1. We can use camera images to establish 2D-2D correspondences:

$$(\tilde{y}_{1,k}, \tilde{y}_{2,k}) \text{ for } k = 1, \dots, N$$



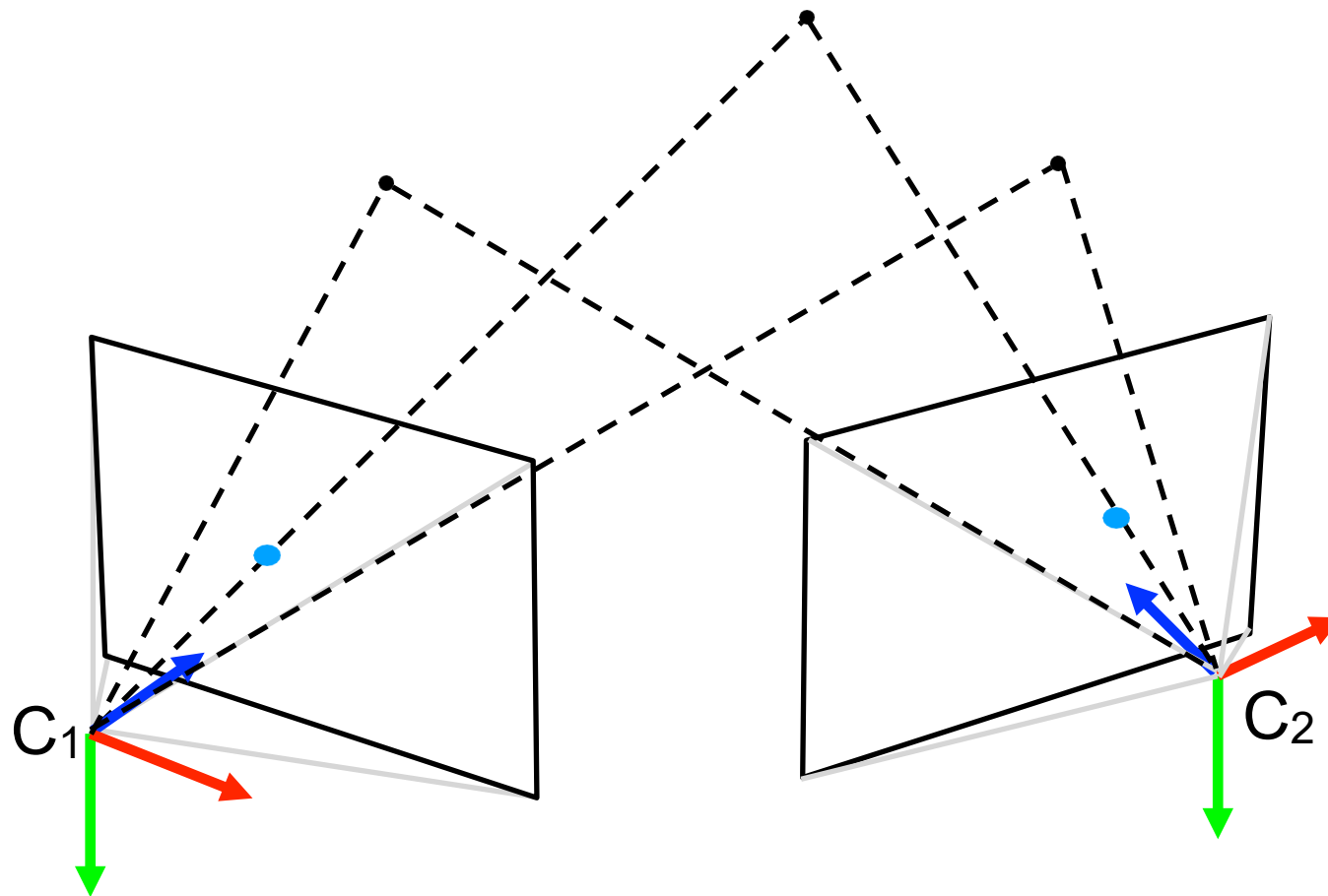
2. For each camera we can compute the set of 3D points corresponding to pixels

We obtain 3D-3D correspondences, which means we have depth information of the point:



$$(p_{1,k}, p_{2,k}) \quad k = 1, \dots, N$$

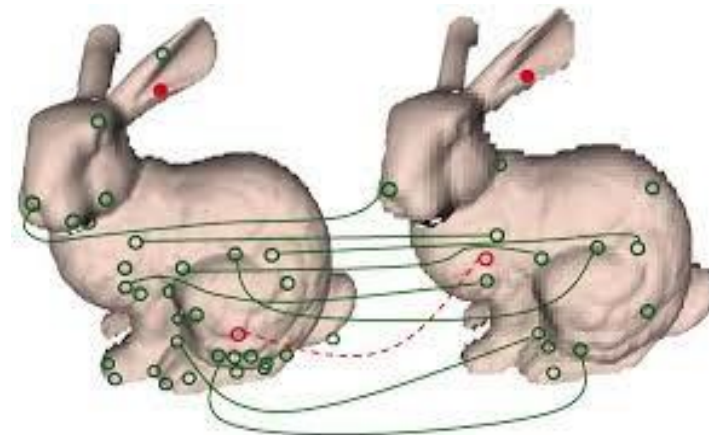
2-view Geometry from 3D-3D Correspondences



How to estimate the relative pose between the cameras from 3D-3D correspondences $(p_{1,k}, p_{2,k})$ with $k = 1, \dots, N$?

Few More Comments:

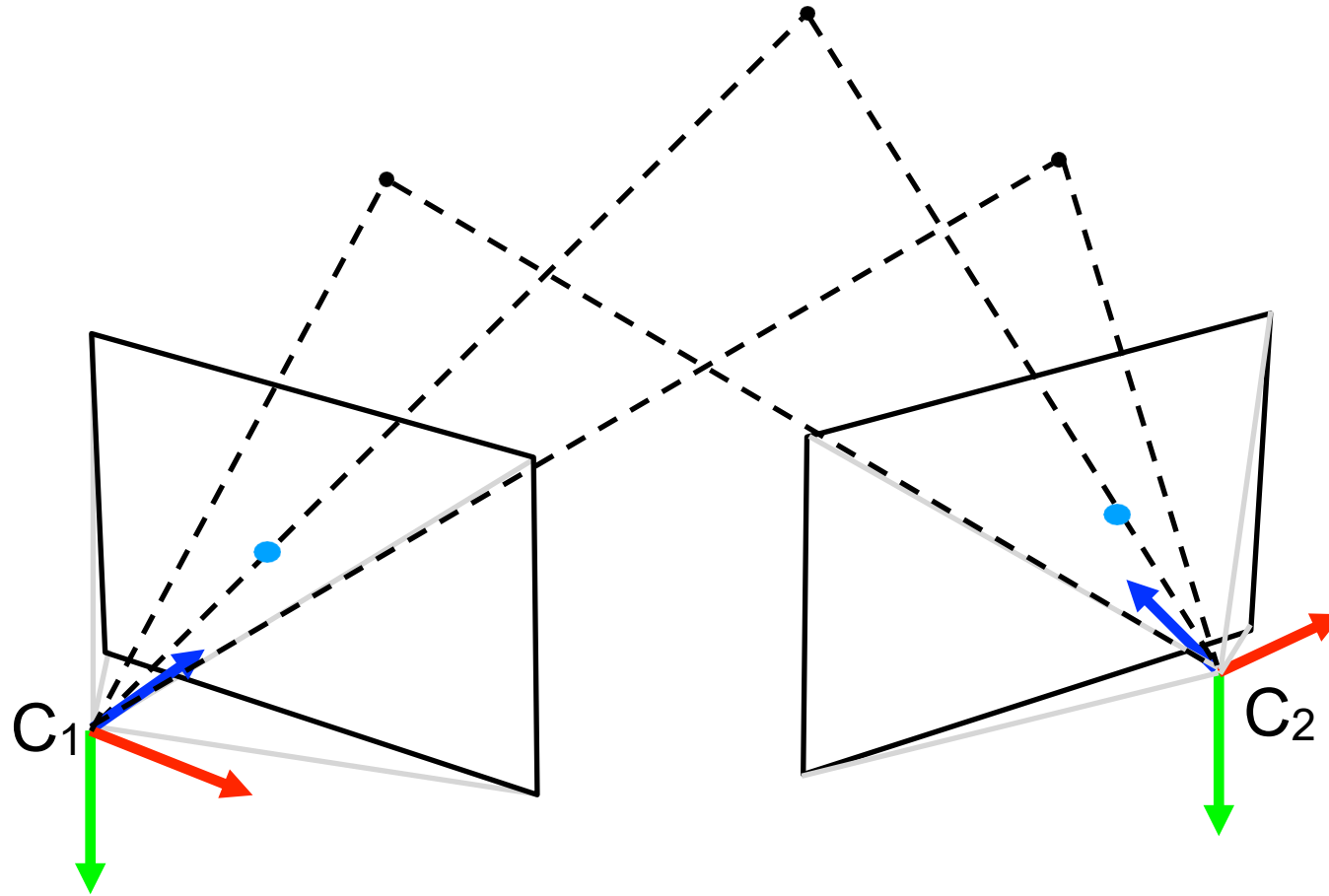
- **3 points** are sufficient to compute the relative pose from 3D-3D correspondences
- We can use the solver seen today as a 3-point minimal solver within a **RANSAC** method
 - Also useful for 3D objects localization:



© Umberto Castellani et al. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Other names: vector registration, point cloud alignment, .. 30

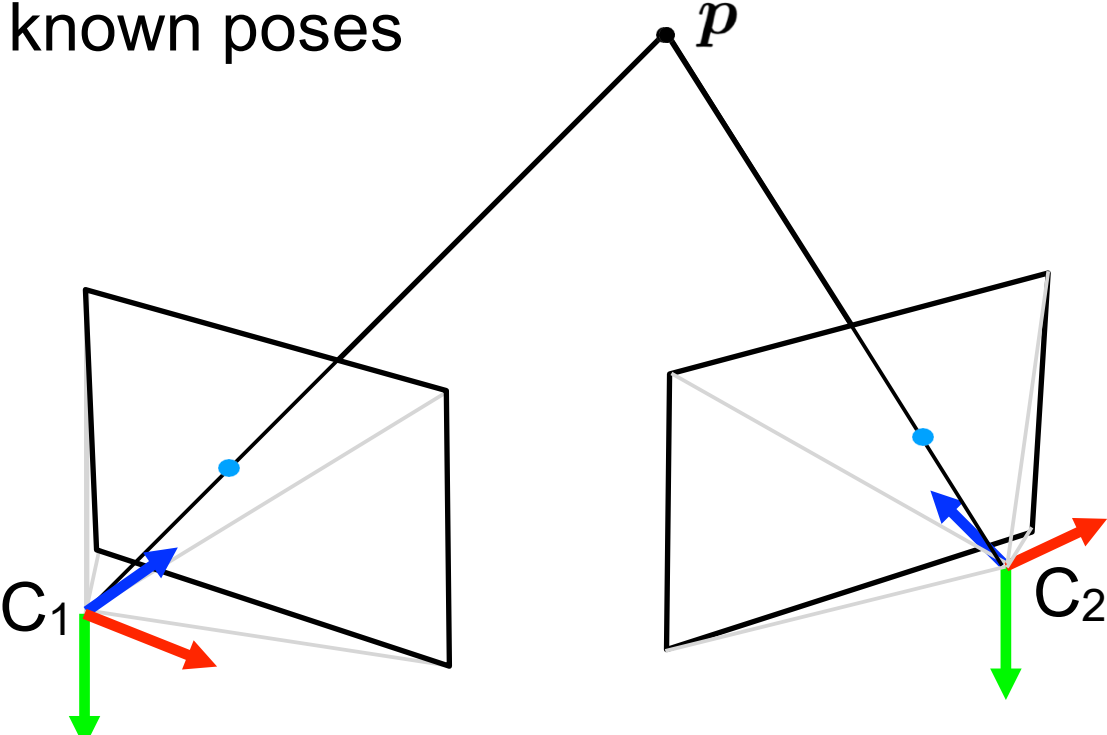
Recap: 2-view Geometry from 3D-3D Correspondences



How to estimate the relative pose between the cameras from 3D-3D correspondences $(p_{1,k}, p_{2,k})$ with $k = 1, \dots, N$?

Example **1a**: Triangulation (Structure Reconstruction)

Compute 3D point from known poses



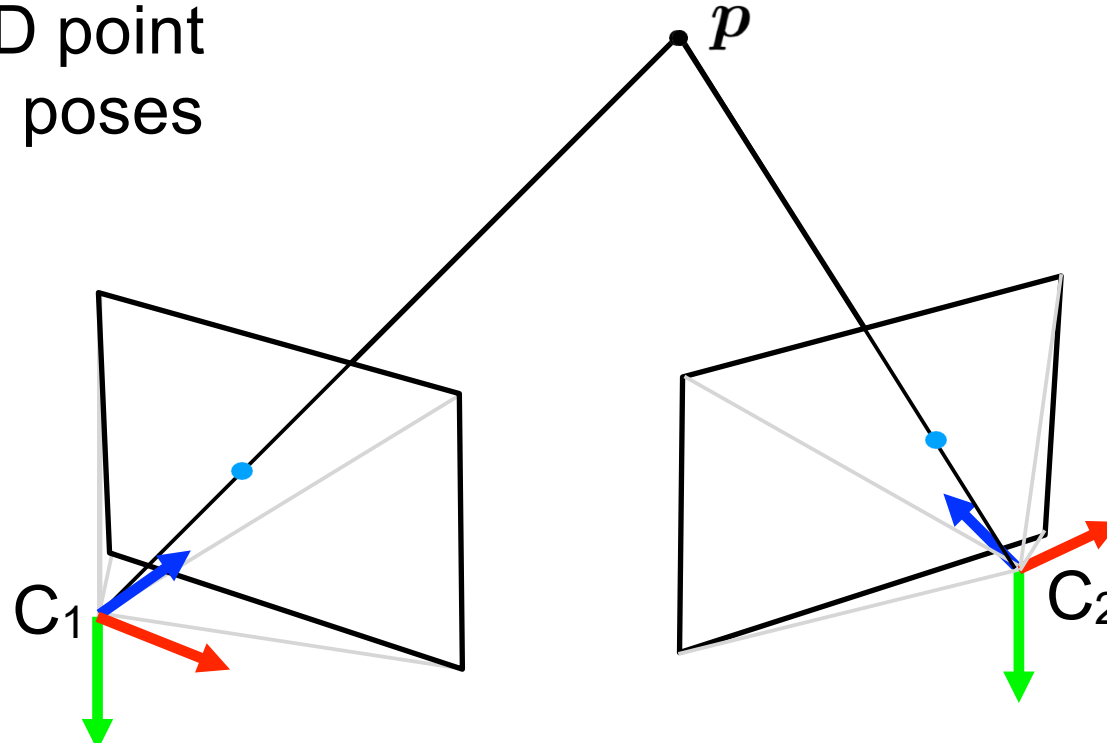
The diagram illustrates the geometry of linear triangulation. A 3D point p is shown at the top. Two cameras, C_1 and C_2 , are positioned below it. Camera C_1 on the left has a green arrow pointing down, a red arrow pointing right, and a blue arrow pointing towards the image plane. Camera C_2 on the right has a green arrow pointing down, a red arrow pointing right, and a blue arrow pointing towards the image plane. Lines connect p to the centers of the image planes in both cameras. The image planes are represented by light gray rectangles. Below each camera, there is a corresponding camera model equation.

$$\lambda_1 \tilde{\mathbf{x}}_1 = \underbrace{\mathbf{K}_1 [\mathbf{R}_w^{c_1} \mathbf{t}_w^{c_1}]}_{\Pi_1} \tilde{\mathbf{p}}^w$$
$$\lambda_2 \tilde{\mathbf{x}}_2 = \underbrace{\mathbf{K}_2 [\mathbf{R}_w^{c_2} \mathbf{t}_w^{c_2}]}_{\Pi_2} \tilde{\mathbf{p}}^w$$

Linear triangulation: $\min_{\|\tilde{\mathbf{p}}^w\|=1} \|\mathbf{A} \tilde{\mathbf{p}}^w\|^2$

Example **1b**: Triangulation (Structure Reconstruction)

Compute 3D point
from known poses

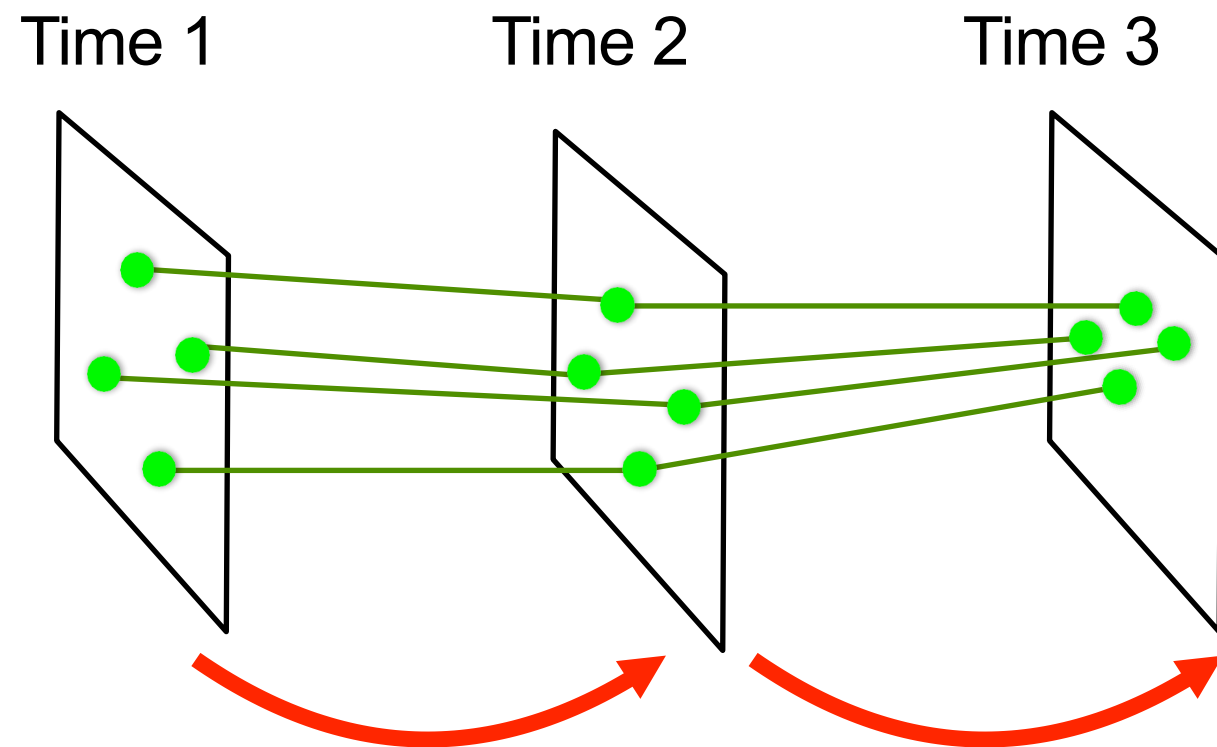


$$\lambda_1 \tilde{x}_1 = \underbrace{K_1 [R_w^{c_1} \ t_w^{c_1}]}_{\Pi_1} \tilde{p}^w$$

$$\lambda_2 \tilde{x}_2 = \underbrace{K_2 [R_w^{c_2} \ t_w^{c_2}]}_{\Pi_2} \tilde{p}^w$$

$$\min_{p^w} \|x_1 - \pi(R_{c_1}^w, t_{c_1}^w, p^w)\|^2 + \|x_2 - \pi(R_{c_2}^w, t_{c_2}^w, p^w)\|^2$$

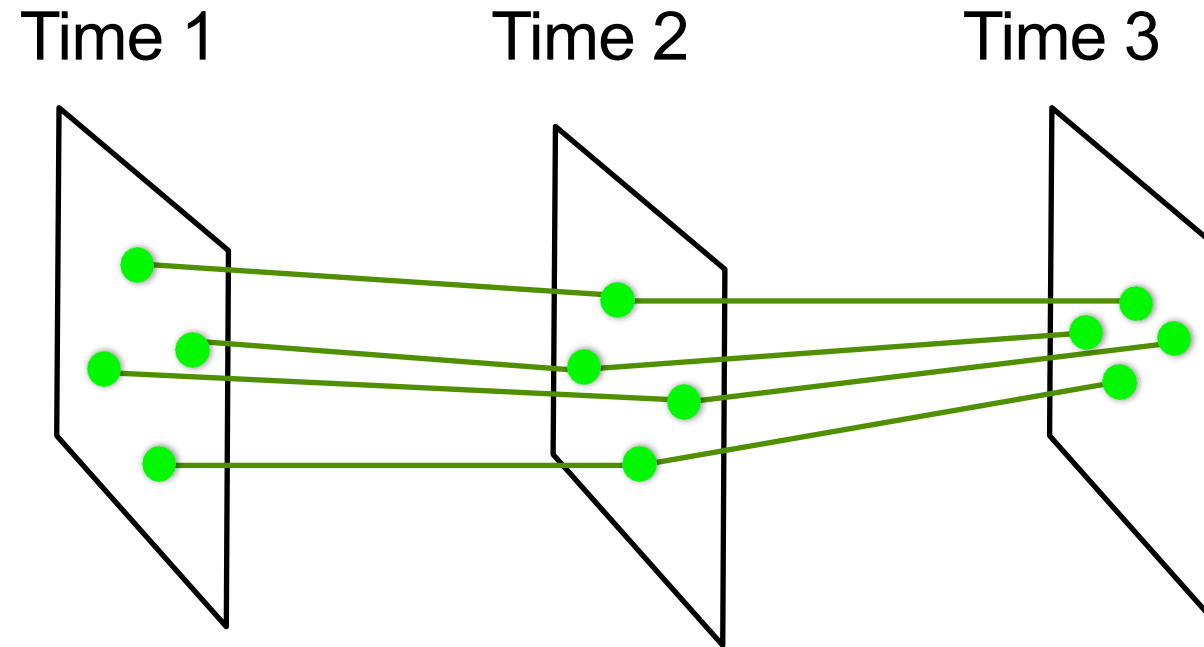
Example **2a**: Motion Estimation



$$\mathbf{E}_{12} = \arg \min_{\mathbf{E}_{12} \in \mathcal{S}_E} \sum_{k=1}^N |\tilde{\mathbf{y}}_{k,2}^T \mathbf{E}_{12} \tilde{\mathbf{y}}_{k,1}|^2$$

$$\mathbf{E}_{23} = \arg \min_{\mathbf{E}_{23} \in \mathcal{S}_E} \sum_{k=1}^N |\tilde{\mathbf{y}}_{k,3}^T \mathbf{E}_{23} \tilde{\mathbf{y}}_{k,2}|^2$$

Example **2b**: Motion Estimation



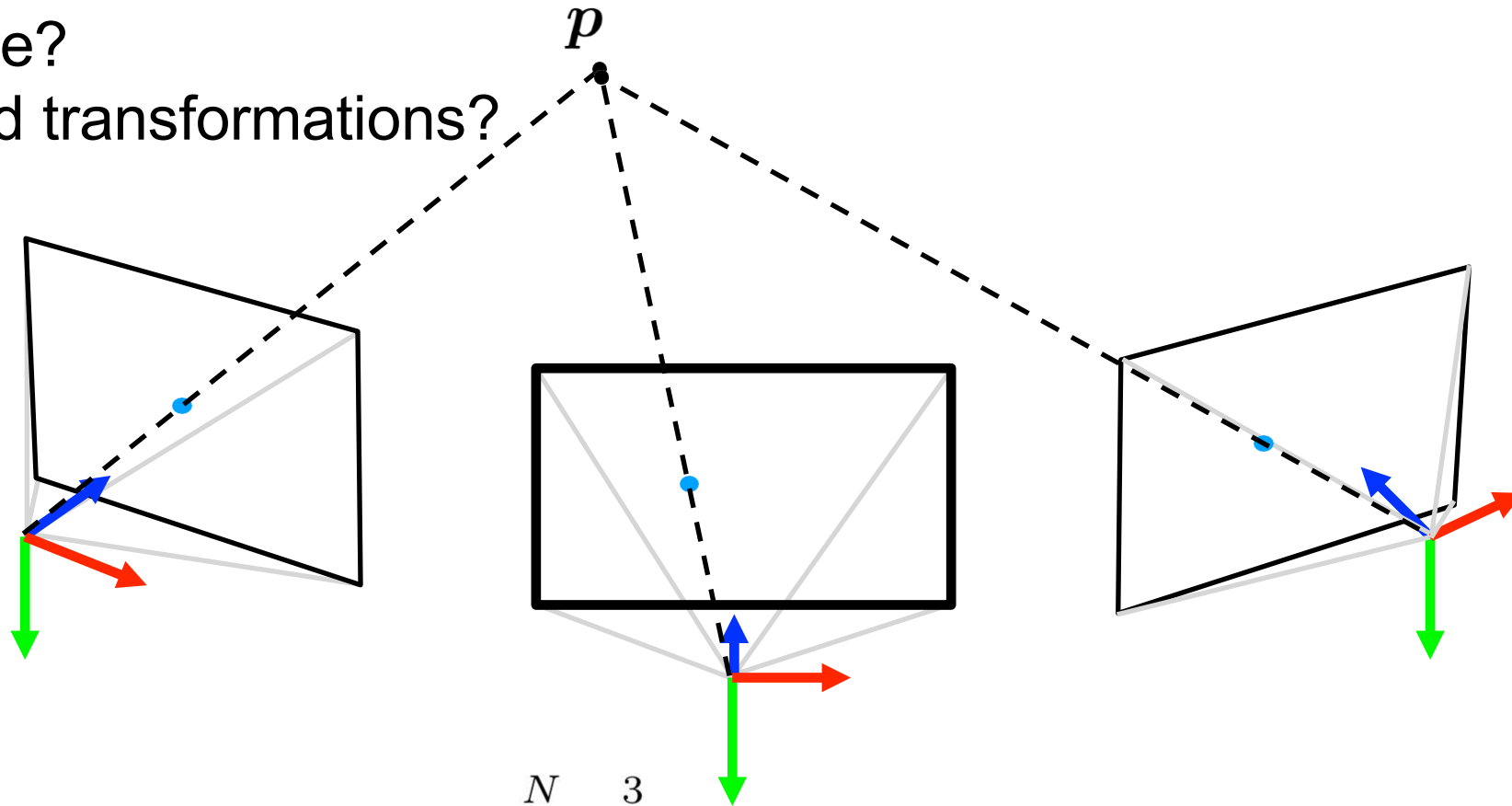
$$\min_{\substack{(\mathbf{R}_{c_i}^w, \mathbf{t}_{c_i}^w), i=1,2,3 \\ \mathbf{p}_k^w, k=1,\dots,N}} \sum_{k=1}^N \sum_{i=1}^3 \|\mathbf{x}_{k,i} - \pi(\mathbf{R}_{c_i}^w, \mathbf{t}_{c_i}^w, \mathbf{p}_k^w)\|^2$$

Generalizes to K cameras: **Bundle adjustment**

Example **2b**: Motion **and Structure** Estimation

Scale?

Rigid transformations?



$$\min_{\substack{(\mathbf{R}_{c_i}^w, \mathbf{t}_{c_i}^w), i=1,2,3 \\ \mathbf{p}_k^w, k=1,\dots,N}} \sum_{k=1}^N \sum_{i=1}^3 \|\mathbf{x}_{k,i} - \pi(\mathbf{R}_{c_i}^w, \mathbf{t}_{c_i}^w, \mathbf{p}_k^w)\|^2$$

Generalizes to K cameras: **Bundle adjustment**

Optimization

Linear triangulation:

$$\min_{\|\tilde{\mathbf{p}}^w\|=1} \|\mathbf{A}\tilde{\mathbf{p}}^w\|^2$$



Nonlinear triangulation:

$$\begin{aligned} \min_{\mathbf{p}^w} & \|\mathbf{x}_1 - \pi(\mathbf{R}_{c_1}^w, \mathbf{t}_{c_1}^w, \mathbf{p}^w)\|^2 + \\ & + \|\mathbf{x}_2 - \pi(\mathbf{R}_{c_2}^w, \mathbf{t}_{c_2}^w, \mathbf{p}^w)\|^2 \end{aligned}$$



Structure from motion (SfM)

